

Benchmark de Algoritmos de Busca

Édipo Elder Fernandes de Melo

20/09/2026

1 Introdução

Este documento apresenta uma análise comparativa de oito algoritmos de busca aplicados a um `std::vector<int>` ordenado. O objetivo é medir, de forma padronizada, o tempo de execução médio (em nanossegundos) de cada algoritmo em função do tamanho do vetor de busca.

1.1 Os algoritmos avaliados

Busca Linear (Iterativa e Recursiva). É o algoritmo mais simples: percorre o vetor elemento por elemento até encontrar o alvo ou atingir o fim. A versão recursiva, dependendo os parâmetros utilizados com o compilador g++, pode gerar um estouro de pilha dependendo do tamanho do vetor utilizando na análise.

Busca Linear Alternativa (Iterativa e Recursiva). Também conhecida como “busca por saltos”. Aproveita a ordenação do vetor para saltar em blocos (no caso, de 2 em 2) e, ao ultrapassar o alvo, retrocede um bloco fazendo varredura linear. Usamos o salto de 2, conforme o slide de referência da disciplina de Estrutura de Dados Básicas I.

Busca Binária (Iterativa e Recursiva). Divide o espaço de busca pela metade a cada iteração. É o algoritmo comumente utilizado para busca em arrays.

Busca Ternária (Iterativa e Recursiva). Similar ao algoritmo de busca binária, porém divide o intervalo em três partes e descarta um ou dois terços a cada iteração.

1.2 Cenário do experimento

O programa foi compilado com os seguintes parâmetros:

```
g++ -std=c++17 -O2 benchmark.cpp -o bench
```

Em todos os testes:

- O vetor `vec` foi criado com tamanho `END_SIZE` preenchido com valor 1.
- O `target` buscado é 0, inexistente no vetor, forçando o pior caso de cada algoritmo — todos percorrem/exploram o vetor até concluir que o elemento não está presente.
- Os tamanhos de vetor variam de 0 a 500000, em passos de 10000. Contudo, a função de busca linear recursiva, com os parâmetros de compilação sem otimização, tendem a estourar a pilha. Neste caso, recomendamos diminuir o tamanho do vetor e o número de passos.
- Cada medição foi repetida 10 (`TRIALS`) vezes, e calculamos a média dos tempos obtidos.

2 A função de benchmark

A fim de evitar repetição de código para cada algoritmo implementado, definimos uma função de execução de cada algoritmo de forma genérica:

```
using algoritmo_busca = bool (*)(const std::vector<int>&, int, int, int);  
  
inline long benchmark(algoritmo_busca algoritmo,
```

```

        const std::vector<int> &vec,
        int begin, int end, int target) {
auto start = std::chrono::high_resolution_clock::now();
bool pos = algoritmo(vec, begin, end, target);
auto stop = std::chrono::high_resolution_clock::now();

volatile bool dummy = pos;    // impede otimização agressiva
(void) dummy;

return std::chrono::duration_cast<std::chrono::nanoseconds>(stop - start).count();
}

```

A implementação dos algoritmos foi feita:

1. **Com a mesma assinatura de parâmetros** Todas as funções recebem (vec, begin, end, target) e retornam bool. O ponteiro de função algoritmo_busca permite chamá-las polimorficamente, sem ramificações no main.
2. **Medição isolada.** O cronômetro só envolve a chamada da implementação do algoritmo em si. Isso evita que custos de outras partes do programa contaminem os resultados.
3. **Reutilização do mesmo vetor em todos os testes.** O vetor é criado apenas uma vez e o tamanho do teste é limitado nos parâmetros begin e end.

O main cria o vetor para testes. Para cada função registrada na tabela funcoes, foram realizadas 10 repetições para um diferentes tamanhos de busca, gerando um arquivo CSV diretamente a ser utilizado pelo R para construção de gráficos, conforme a seguir:

Carregamos o CSV e o transformamos para o formato longo:

```

tempos <- read.csv("tempos.csv", check.names = FALSE)

# Converte colunas Tamanho_X em linhas
tempos_longo <- tempos |>
  pivot_longer(
    cols = starts_with("Tamanho_"),
    names_to = "Tamanho",
    values_to = "Tempo_ns"
  ) |>
  mutate(
    Tamanho = as.integer(str_remove(Tamanho, "Tamanho_"))
  )

# Função auxiliar para plotar um algoritmo específico
plotar_algoritmo <- function(nome, cor = "steelblue") {
  dados <- filter(tempos_longo, Algoritmo == nome)
  ggplot(dados, aes(x = Tamanho, y = Tempo_ns)) +
    geom_line(color = cor, linewidth = 1) +
    geom_point(color = cor, size = 1.5) +
    labs(
      title = nome,
      x = "Tamanho do vetor",
      y = "Tempo (ns)"
    ) +
    theme_minimal(base_size = 12)
}

```

3 Análise por algoritmo

3.1 Busca Linear Iterativa

Percebe-se que crescimento é visivelmente linear. As variações observáveis ao longo da reta são consequências da execução no sistema operacional.

```
bool busca_linear_iterativa(const std::vector<int> &vec, int begin, int end, int target) {  
    for (int i = begin; i < end; i++) {  
        if (vec[i] == target) return true;  
    }  
    return false;  
}
```

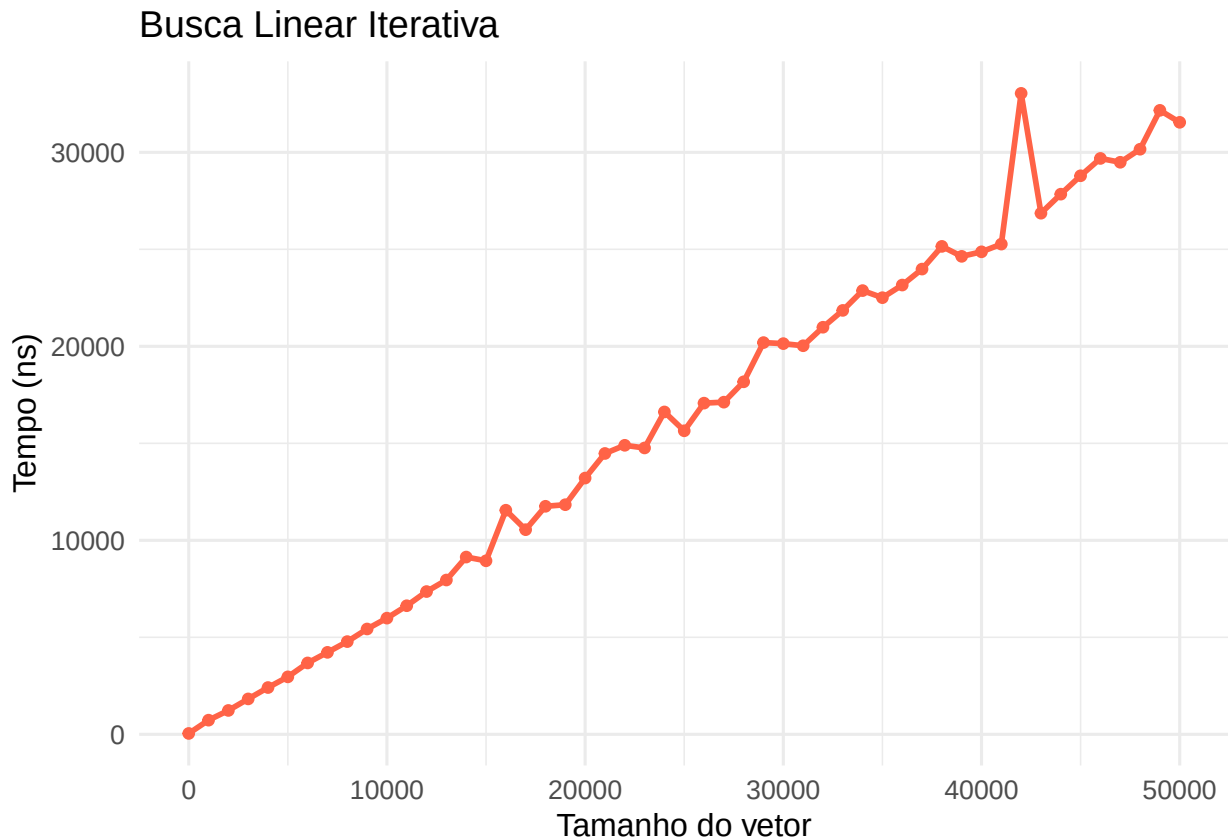


Figura 1: Busca Linear Iterativa

3.2 Busca Linear Recursiva

Mesma ideia, mas recursiva. Cada chamada da função empilha um novo *stack frame*. Para vetores maiores, sem a utilização de parâmetros adequados no compilador, um estouro de pilha ocorre e a execução do programa é interrompida.

```
bool busca_linear_recursiva(const std::vector<int> &vec, int idx, int end, int target) {  
    if (idx >= end) return false;           // limite ANTES de acessar vec[idx]  
    if (vec[idx] == target) return true;  
    return busca_linear_recursiva(vec, idx + 1, end, target);  
}
```

3.3 Busca Linear Alternativa Iterativa

O algoritmo funciona saltando de 2 em 2 posições enquanto `vec[i] < target` e depois recua um bloco varrendo de 1 em 1.

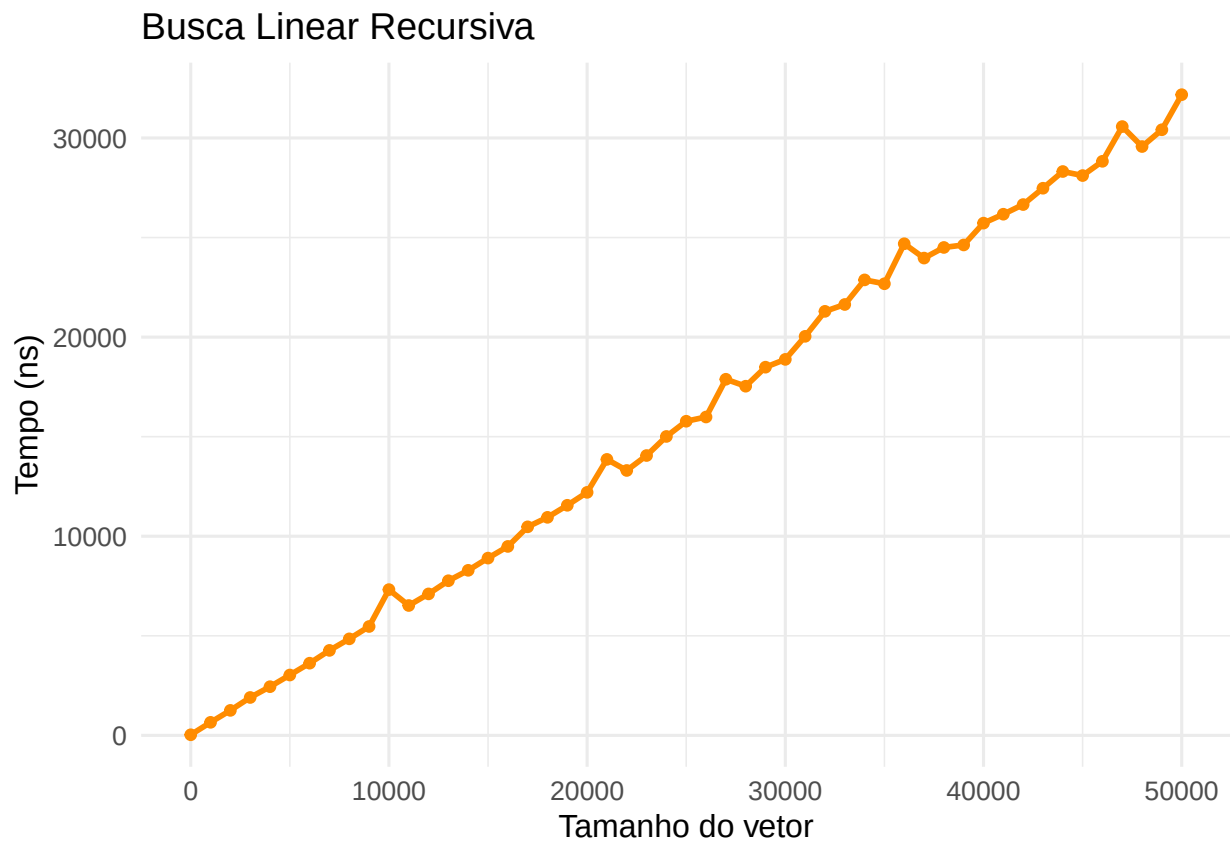


Figura 2: Busca Linear Recursiva

```
bool busca_linear_alternativa_iterativa(const std::vector<int> &vec, int begin, int end, int target)
{
    int i = begin;
    while (i < end && vec[i] < target) i += SALTO;
    for (int j = 0; j < SALTO && i - j >= begin; j++) {
        if (vec[i - j] == target) return true;
    }
    return false;
}
```

3.4 Busca Linear Alternativa Recursiva

Versão recursiva do *jump search* introduzimos a condição de parada `if (begin >= end) return false;` na primeira linha.

```
bool busca_linear_alternativa_recursiva_salto(const std::vector<int> &vec,
                                              int begin, int end,
                                              int salto, int target) {

    if (begin >= end) return false;

    if (vec[begin] < target) {
        int proximo = begin + salto;
        if (proximo >= end) proximo = end - 1;
        return busca_linear_alternativa_recursiva_salto(vec, proximo, end, salto, target);
    }

    for (int j = 0; j < salto && begin - j >= 0; ++j) {
        if (vec[begin - j] == target) return true;
    }
}
```

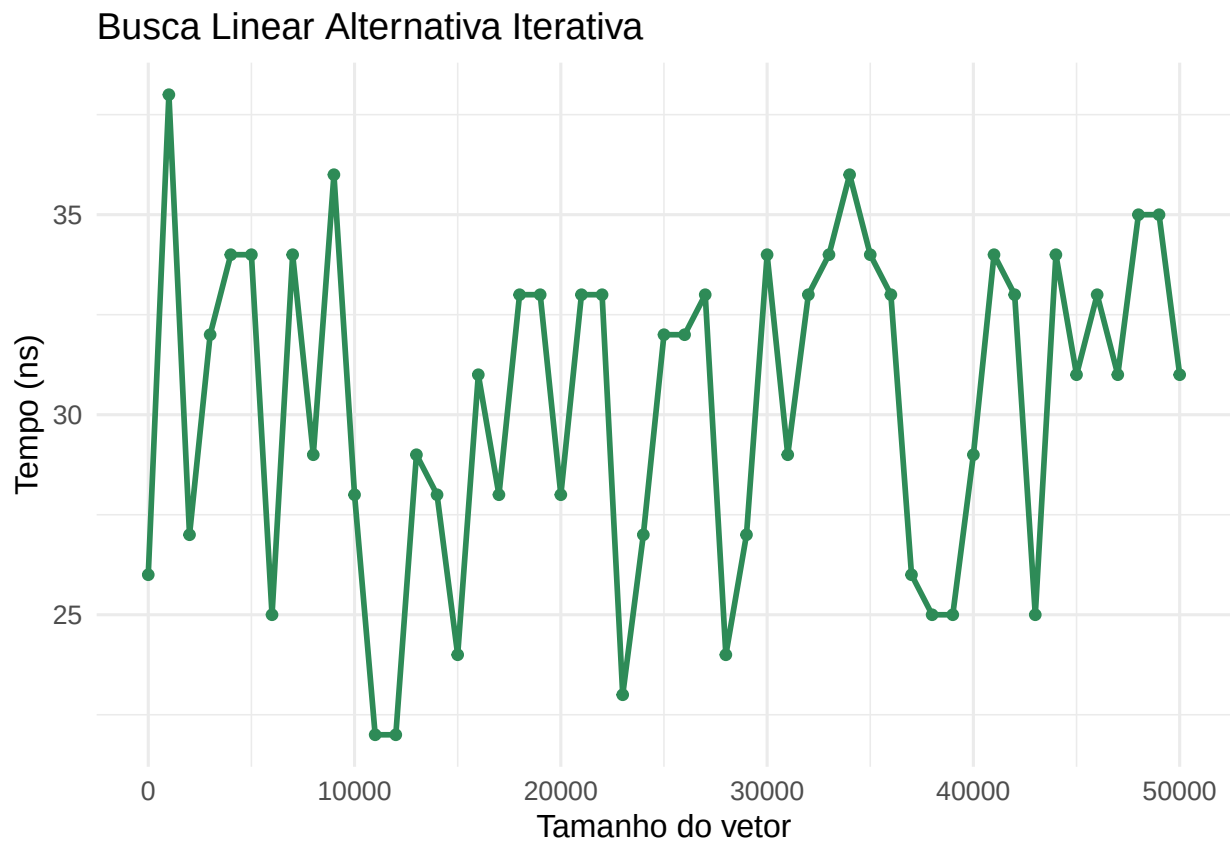


Figura 3: Busca Linear Alternativa Iterativa

```
return false;
}
```

3.5 Busca Binária Iterativa

Algoritmo mais comum em implementações de buscas em arrays.

```
bool busca_binaria_iterativa(const std::vector<int> &vec, int begin, int end, int target) {
    while (begin <= end) {
        int idx = begin + (end - begin) / 2;
        if (vec[idx] == target) return true;
        else if (vec[idx] > target) end = idx - 1;
        else begin = idx + 1;
    }
    return false;
}
```

Note a curva praticamente plana: o tempo não cresce de forma linear.

3.6 Busca Binária Recursiva

Equivalente algorítmico da versão iterativa, apenas reescrito para fazer chamadas recursivas.

```
bool busca_binaria_recursiva(const std::vector<int> &vec, int begin, int end, int target) {
    if (begin > end) return false;
    int idx = begin + (end - begin) / 2;
    if (vec[idx] == target) return true;
    else if (vec[idx] > target) return busca_binaria_recursiva(vec, begin, idx - 1, target);
    else return busca_binaria_recursiva(vec, idx + 1, end, target);
}
```

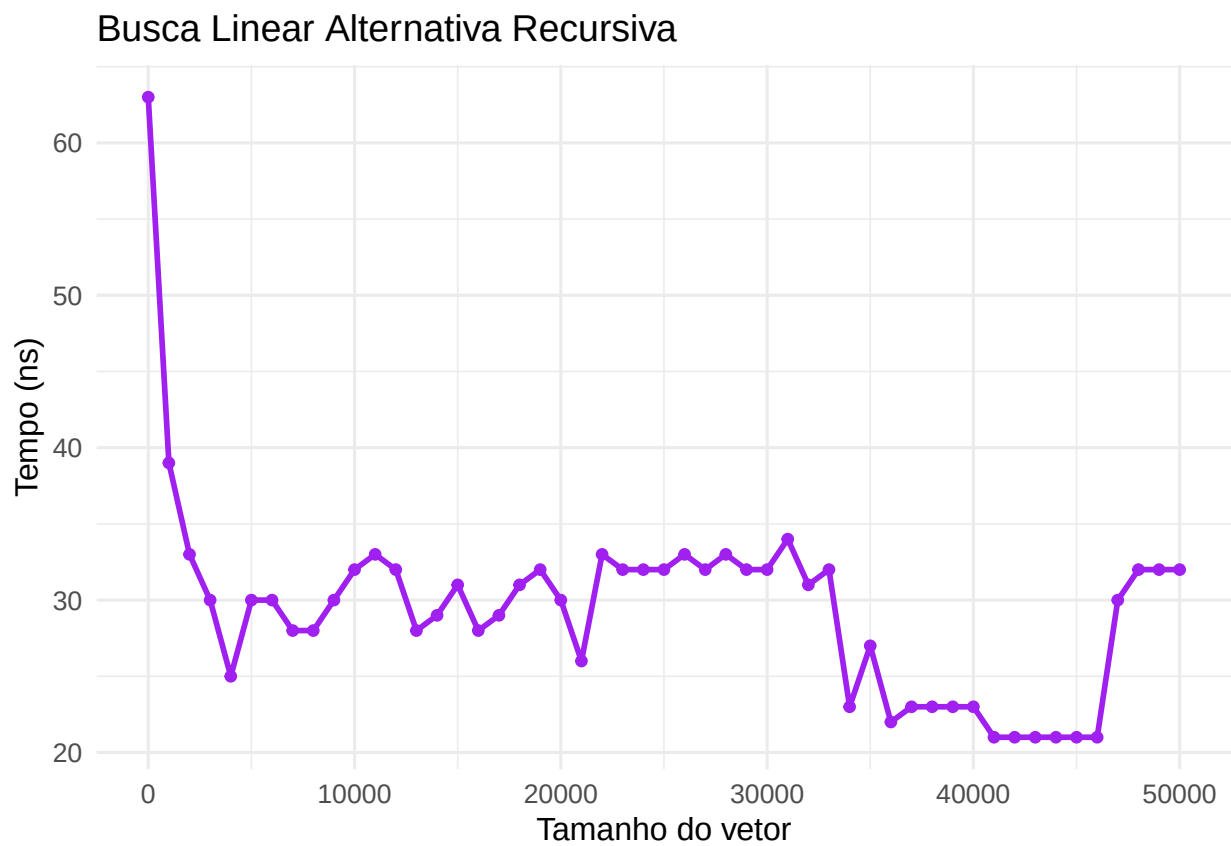


Figura 4: Busca Linear Alternativa Recursiva

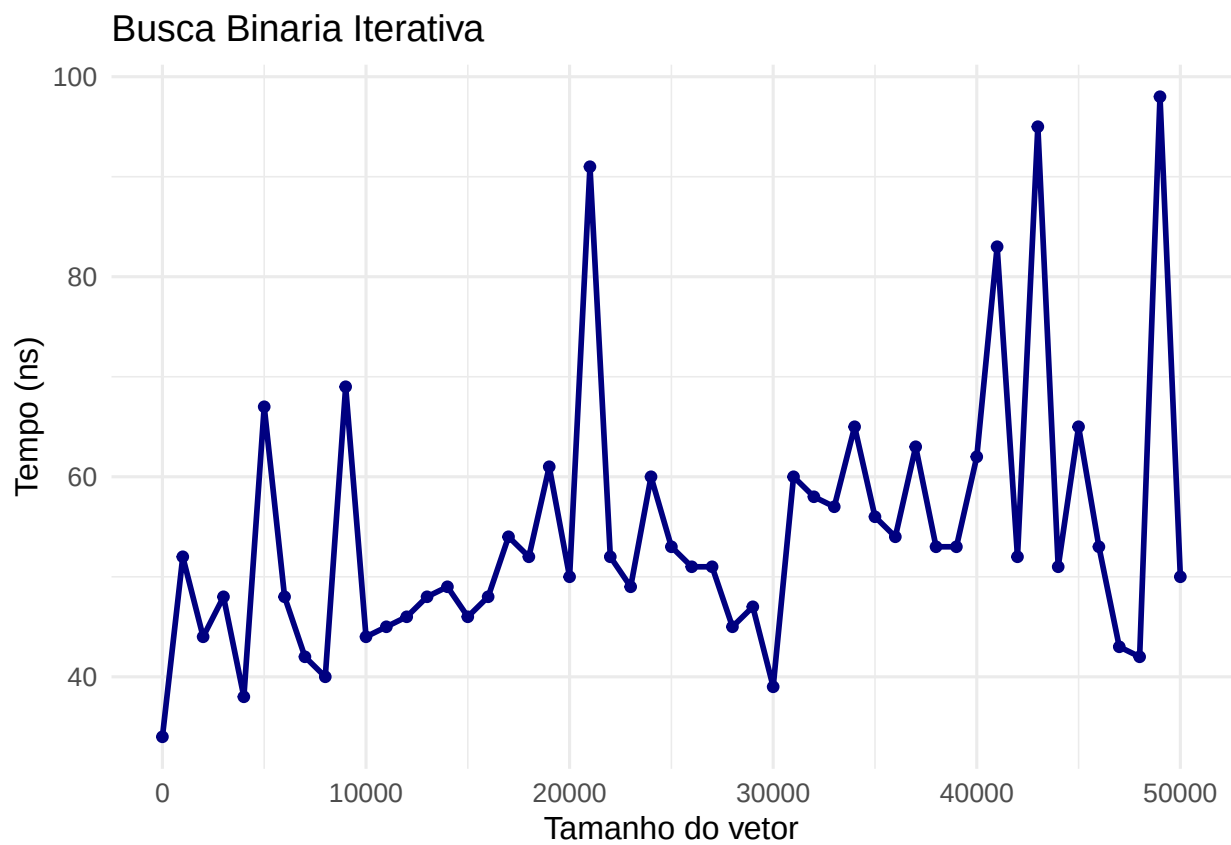


Figura 5: Busca Binária Iterativa

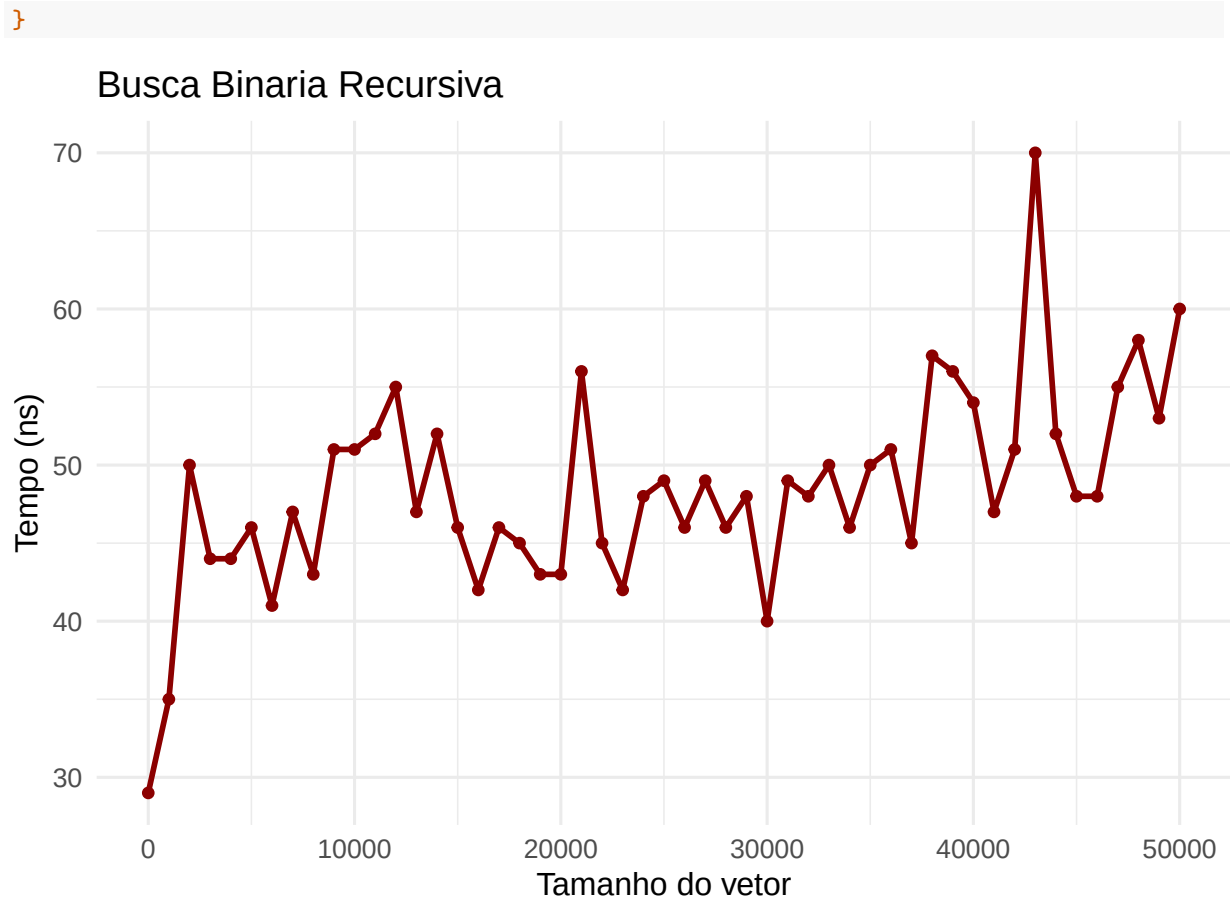


Figura 6: Busca Binária Recursiva

Praticamente indistinguível da iterativa em tempo — o *overhead* de empilhamento de chamadas é quase nulo.

3.7 Busca Ternária Iterativa

Divide o vetor em três partes. Apesar de reduzir o intervalo por um fator de 3 (em vez de 2), faz duas comparações por iteração (*terco_1* e *terco_2*). O número de iterações é menor, mas o custo comparação em C++ anula os ganhos de tempo que seriam obtidos com menos iterações.

```
bool busca_ternaria_iterativa(const std::vector<int>& vec, int begin, int end, int target) {
    while (begin <= end) {
        int terco_1 = begin + (end - begin) / 3;
        int terco_2 = begin + 2 * (end - begin) / 3;

        if (vec[terco_1] == target || vec[terco_2] == target) return true;
        else if (target < vec[terco_1]) end = terco_1 - 1;
        else if (target > vec[terco_2]) begin = terco_2 + 1;
        else { begin = terco_1 + 1; end = terco_2 - 1; }
    }
    return false;
}
```

3.8 Busca Ternária Recursiva

Mesma lógica da iterativa, apenas reescrito para chamadas recursivas.

```
bool busca_ternaria_recursiva(const std::vector<int>& vec, int begin, int end, int target) {
    if (begin > end) return false;
```

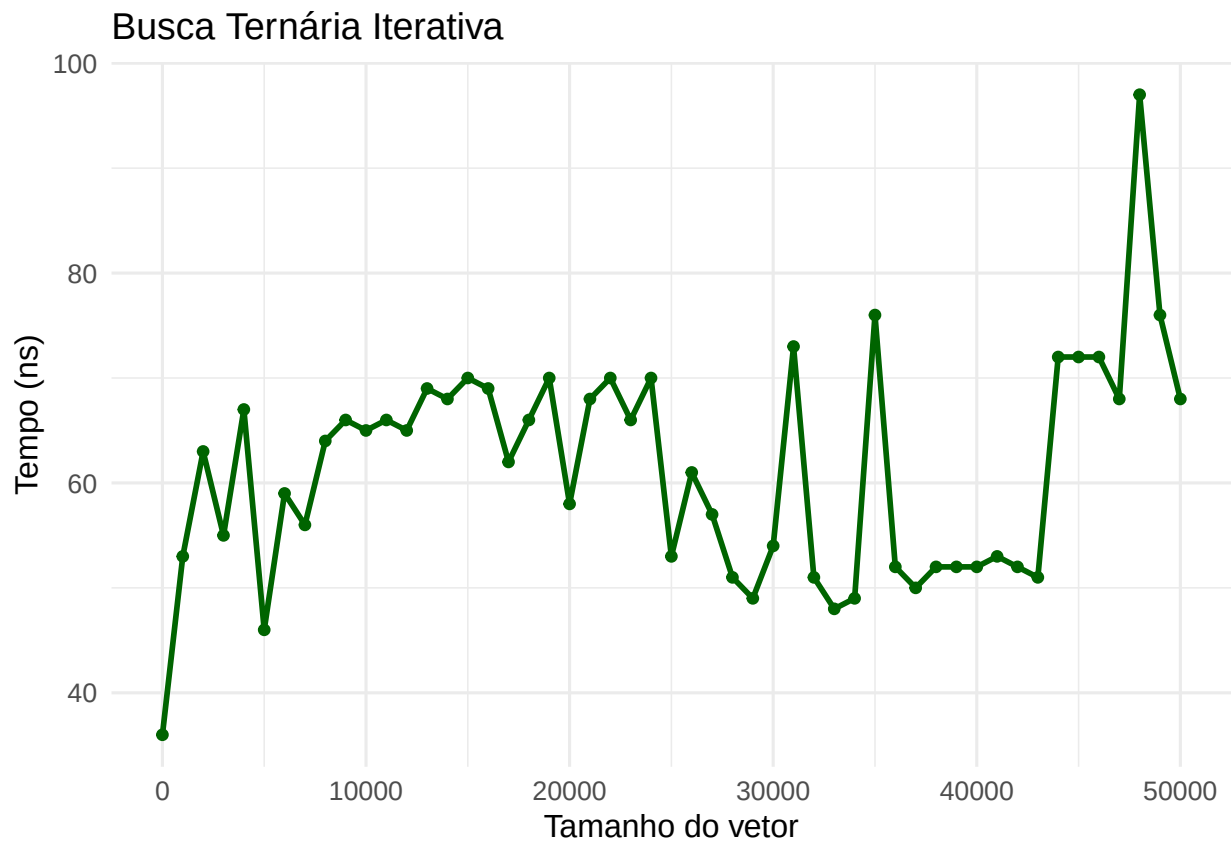


Figura 7: Busca Ternária Iterativa

```
int terco_1 = begin + (end - begin) / 3;
int terco_2 = begin + 2 * (end - begin) / 3;

if (vec[terco_1] == target || vec[terco_2] == target) return true;
else if (target < vec[terco_1]) return busca_ternaria_recursiva(vec, begin, terco_1 - 1, target);
else if (target > vec[terco_2]) return busca_ternaria_recursiva(vec, terco_2 + 1, end, target);
else return busca_ternaria_recursiva(vec, terco_1 + 1, terco_2 - 1, target);
}
```

4 Conclusão

O gráfico abaixo reúne os oito algoritmos na mesma escala, tornando imediata a comparação de ordens de crescimento:

```
ggplot(tempos_longo, aes(x = Tamanho, y = Tempo_ns, color = Algoritmo)) +
  geom_line(linewidth = 1) +
  labs(
    title = "Comparação de algoritmos de busca",
    subtitle = "Pior caso: alvo inexistente, vetor ordenado de 1s",
    x = "Tamanho do vetor",
    y = "Tempo (ns)",
    color = "Algoritmo"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom") +
  guides(color = guide_legend(nrow = 2))
```

Excluindo os tempos de buscas lineares, podemos compara os algoritmos mais eficientes:

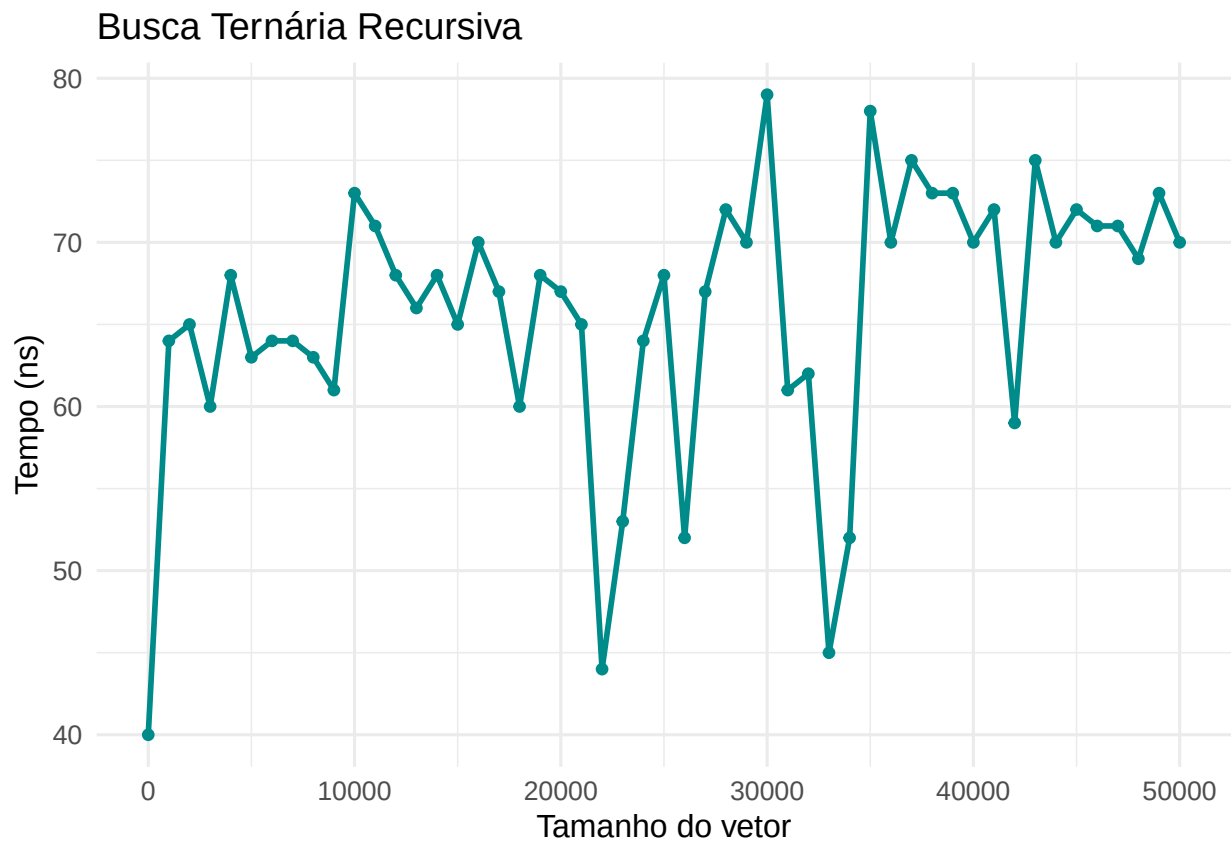


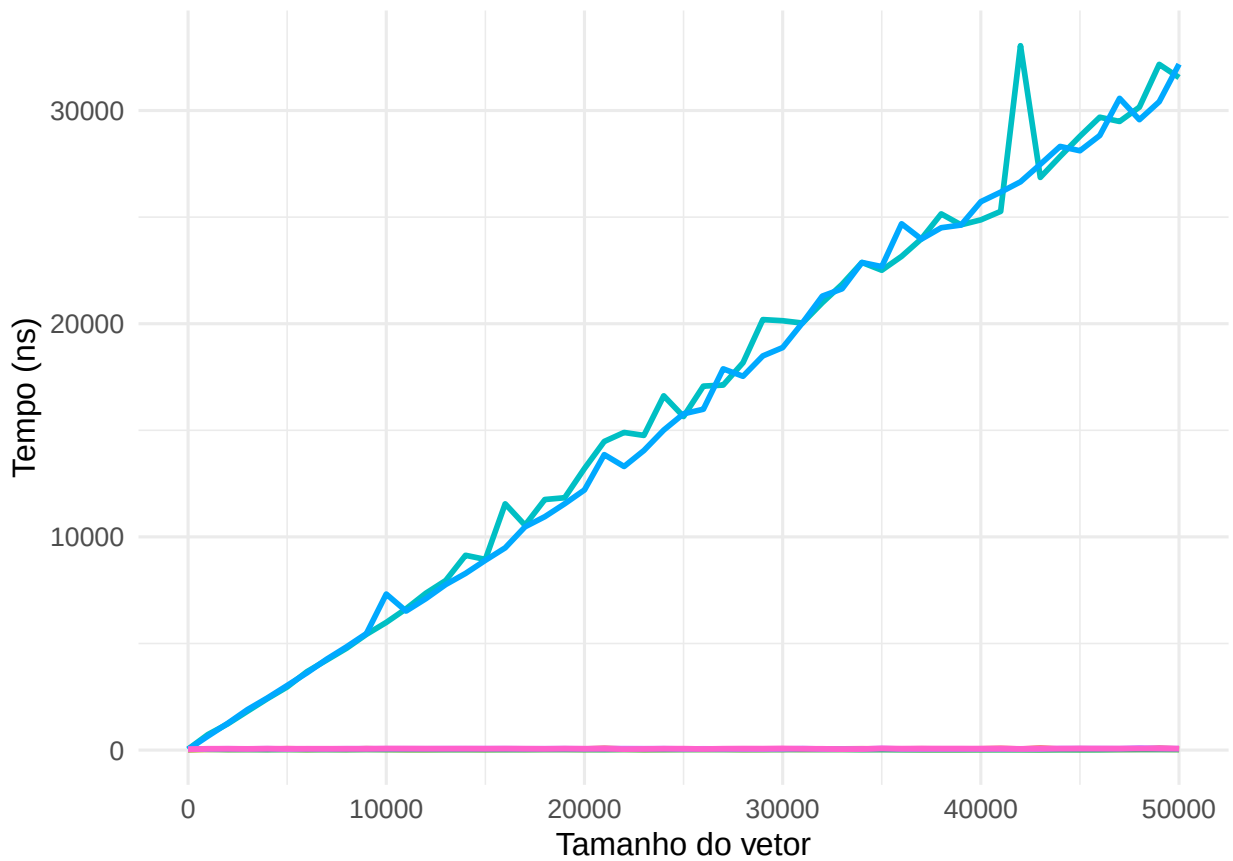
Figura 8: Busca Ternária Recursiva

```
algoritmos_log <- c(
  "Busca Binária Iterativa",
  "Busca Binária Recursiva",
  "Busca Binária Alternativa Iterativa",
  "Busca Binária Alternativa Recursiva",
  "Busca Ternária Iterativa",
  "Busca Ternária Recursiva"
)

tempos_sem_lineares <- tempos_longo |>
  filter(Algoritmo %in% algoritmos_log)

ggplot(tempos_sem_lineares,
  aes(x = Tamanho, y = Tempo_ns, color = Algoritmo)) +
  geom_line(linewidth = 1) +
  labs(
    title = "Comparação sem as buscas lineares",
    subtitle = "Algoritmos de melhor performance (escala ampliada)",
    x = "Tamanho do vetor",
    y = "Tempo (ns)",
    color = "Algoritmo"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom") +
  guides(color = guide_legend(nrow = 2))
```

Comparação de algoritmos de busca Pior caso: alvo inexistente, vetor ordenado de 1s

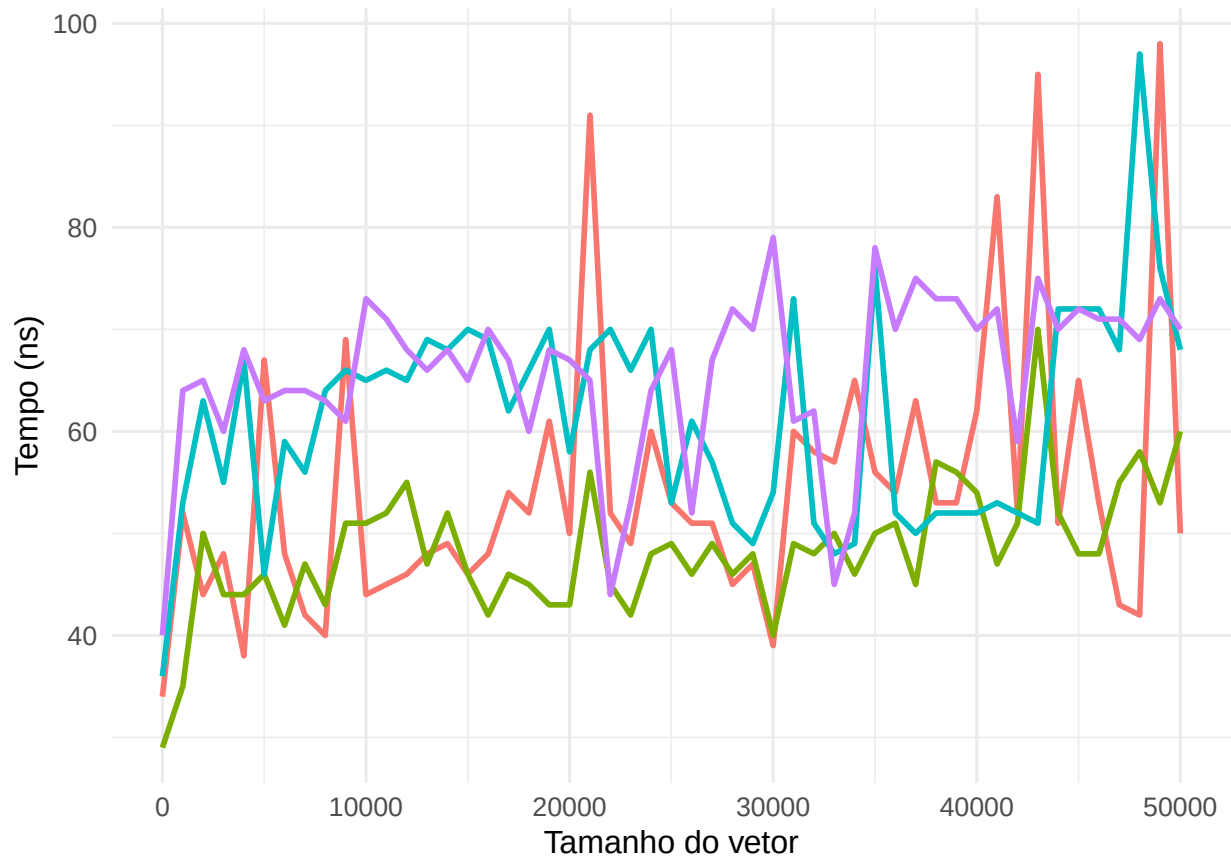


- Busca Binária Iterativa
- Busca Linear Alternativa Iterativa
- Busca Linear Iterativa
- Busca Binária Recursiva
- Busca Linear Alternativa Recursiva
- Busca Linear Recursiva

Figura 9: Comparação entre todos os algoritmos

Comparação sem as buscas lineares

Algoritmos de melhor performance (escala ampliada)



Algoritmo

- Busca Binária Iterativa
- Busca Ternária Iterativa
- Busca Binária Recursiva
- Busca Ternária Recursiva

Figura 10: Comparação entre os algoritmos de melhor performance

4.1 Conclusão

O cenário de pior caso (alvo ausente) é importante para avaliar a eficácia teórica do algoritmo. Contudo, na prática, é mais comum achar o alvo no caso médio, o que daria tempos menores aos obtidos.

A medição com a biblioteca `chronos` não é a ideal, uma vez que o sistema operacional pode flutuar a execução do programa. Isso, inclusive, foi observado ao executar o programa diversas vezes e obtendo gráficos ligeiramente distintos.

Concluindo a análise, comprovamos que, para vetores ordenados, é recomendado utilizar a busca binária.